TECHNICAL STANDARD OPERATING PROCEDURE

Enterprise-Grade PostgreSQL High Availability Cluster

Patroni + etcd + pgBackRest + HAProxy + PgBouncer + Keepalived

Technology / Platform	PostgreSQL HA on Linux
Target Version	PostgreSQL 17; Patroni 4.x reference design
Document Type	Technical Architecture / Operations SOP
Document Version	1.0
Prepared Date	27 August 2026
Classification	Internal / Controlled Technical Document

Purpose: production-oriented implementation, validation, operations, backup/recovery, and failover guidance.

Document Control

Field	Value
Document Title	Enterprise-Grade PostgreSQL High Availability Cluster
Purpose	Define a repeatable PostgreSQL 17 HA implementation and operating model using Patroni, etcd, pgBackRest, HAProxy, PgBouncer, and Keepalived.
Scope	Three PostgreSQL/Patroni nodes, a three-member DCS reference design, connection routing/pooling, backup/recovery, monitoring, security, HA/DR validation, and operational procedures.
Technology / Platform	PostgreSQL 17 on Linux
Target Version	PostgreSQL 17; Patroni 4.x; current supported component releases validated by the OS/package repository before deployment
Document Version	1.0
Primary Documentation Source	Official PostgreSQL, Patroni, pgBackRest, HAProxy, PgBouncer, Keepalived, and etcd documentation
Intended Audience	Database engineers, SREs, platform engineers, Linux administrators, technical architects
Document Owner	Database / Platform Engineering
Classification	Internal / Controlled Technical Document
Review Trigger	Major version upgrade, architecture change, security policy change, backup/DR change, or annual review

Revision History

Version	Date	Description
1.0	27 August 2026	Initial Release

Source and Accuracy Statement

This SOP was prepared and cross-checked against the applicable official product and project documentation listed in the Official References section. The examples are a production-oriented reference design, not a literal guarantee that every value fits every environment.

Before production use, validate all commands and settings against the exact PostgreSQL, Patroni, operating system, package versions, network design, storage, DCS, extensions, security requirements, workload profile, RPO/RTO, and organizational standards.

Significant design note: PostgreSQL 17 is the database target. Patroni 4.x is used as the current reference family. Exact package versions for HAProxy, PgBouncer, Keepalived, pgBackRest, and etcd should be pinned to releases supported by the chosen operating system and organizational lifecycle policy.

TABLE OF CONTENTS / INDEX

1. Introduction

1.1 Purpose

This SOP defines a production-oriented PostgreSQL high-availability architecture built around Patroni-managed PostgreSQL streaming replication. It adds a distributed configuration store for leader coordination, pgBackRest for backup and replica creation, HAProxy for role-aware routing, PgBouncer for connection pooling, and Keepalived for a floating client endpoint.

1.2 Objectives

- Maintain one writable PostgreSQL primary and two streaming replicas across three database nodes.
- Automate leader election and controlled failover with Patroni while avoiding application-side node awareness.
- Provide a stable virtual IP (VIP) and connection pool endpoint for applications.
- Use pgBackRest for full/incremental/differential backups, WAL archiving, restore, and optional replica bootstrap.
- Provide repeatable validation, switchover, failover, recovery, monitoring, security, and troubleshooting procedures.
- Separate lab convenience from production controls such as TLS, firewalling, least privilege, monitoring, capacity planning, and tested restore procedures.

1.3 Reference Node Plan

Role	Hostname	Example IP	Key Services
DB Node 1	pgn1	70.80.90.110	PostgreSQL, Patroni, etcd, HAProxy, PgBouncer, Keepalived
DB Node 2	pgn2	70.80.90.111	PostgreSQL, Patroni, etcd, HAProxy, PgBouncer, Keepalived
DB Node 3	pgn3	70.80.90.112	PostgreSQL, Patroni, etcd, HAProxy, PgBouncer, Keepalived
Client VIP	pg-vip	70.80.90.100	Floating endpoint; replace with an address reserved for the target subnet

The IP addresses above are examples based on the supplied environment. Treat them as placeholders until network ownership, subnet, routing, firewall, and VIP reservation are formally confirmed.

2. Architecture

2.1 Logical Architecture

```
Applications / Services
    |
    v
Virtual IP (Keepalived / VRRP)
    |
    v
PgBouncer :6432
    |
    v
```

```
HAProxy write endpoint :5000  ----> Patroni REST health check /primary :8008
    |                       |
    +-------------------------------+
    |
    +----> pgn1 PostgreSQL :5432
    +----> pgn2 PostgreSQL :5432
    +----> pgn3 PostgreSQL :5432
           |
           +---- PostgreSQL streaming replication / WAL
           |
           +---- Patroni <----> etcd quorum (DCS)
           |
           +---- pgBackRest <----> backup repository / object storage
```

2.2 Component Responsibilities

Component	Responsibility	Production Design Point
PostgreSQL 17	Database engine and physical streaming replication	Use durable storage, WAL archiving, tuned memory/I/O, SCRAM authentication, TLS where required.
Patroni	Leader election, PostgreSQL lifecycle, failover/switchover orchestration	All PostgreSQL service control should go through Patroni; avoid manually starting a conflicting postmaster.
etcd	Distributed consensus / cluster state for Patroni	Use an odd-sized quorum, TLS, backups, and failure-domain awareness.
pgBackRest	Backups, WAL archive, restore, optional replica imaging	Repository must be independently protected and restore-tested.
HAProxy	Role-aware TCP routing	Use Patroni REST endpoints so only the current primary receives write traffic.
PgBouncer	Connection pooling	Transaction pooling is efficient but requires application compatibility testing.
Keepalived	VIP ownership via VRRP	Track the local HAProxy/PgBouncer service and prevent a failed proxy node from retaining the VIP.

2.3 Data, Control, and Network Flow

- Data flow: application -> VIP -> PgBouncer -> HAProxy -> current PostgreSQL primary.
- Replication flow: primary WAL -> streaming replicas; archived WAL -> pgBackRest repository.
- Control flow: Patroni instances coordinate through etcd; Patroni promotes/demotes PostgreSQL according to DCS state and health.
- Routing flow: HAProxy queries Patroni REST /primary or /read-write. A node returns HTTP 200 only when it is the primary holding the leader lock.

- VIP flow: Keepalived elects one proxy node to own the VIP. If its tracked service fails, priority/state changes allow another node to take the address.

2.4 Failure Domains

A three-node design tolerates loss of one PostgreSQL node while retaining two database members. A three-member etcd cluster also retains quorum after one etcd member fails. Co-locating database, DCS, proxy, and VIP services on the same three servers is compact, but a production design should assess correlated failure: a single host outage removes one member from several layers at once. For higher isolation, place the DCS and client proxy tier on independent failure domains.

3. Prerequisites

3.1 Platform Prerequisites

☐ Three Linux hosts with stable hostnames, static IP addresses, synchronized time, and forward/reverse name resolution.

☐ PostgreSQL 17 packages and repository approved for the operating system.

☐ Patroni 4.x package or Python environment with a supported PostgreSQL driver and etcd client support.

☐ Three-member etcd cluster or another Patroni-supported DCS; this SOP uses etcd as the reference.

☐ pgBackRest installed on database nodes and repository host as required by the selected repository topology.

☐ HAProxy, PgBouncer, and Keepalived packages installed on the proxy/VIP nodes.

☐ Dedicated PostgreSQL data filesystem with capacity, IOPS, latency, filesystem, mount options, and backup retention sized for the workload.

☐ Dedicated backup repository or object storage with capacity for retention plus WAL growth and restore testing.

☐ Firewall rules approved for only required source/destination pairs.

☐ DNS/VIP ownership, monitoring, alert routing, backup RPO/RTO, maintenance windows, and escalation contacts approved.

3.2 Reference Ports

Port	Protocol	Purpose	Restrict To
5432	TCP	PostgreSQL	HAProxy/PgBouncer, DBA/admin, replication peers as required
6432	TCP	PgBouncer client endpoint	Application networks
5000	TCP	HAProxy write endpoint	PgBouncer/proxy tier
5001	TCP	Optional HAProxy read-only endpoint	Read-only clients/pools
8008	TCP/HTTP(S)	Patroni REST API	Cluster/proxy/monitoring networks only
2379	TCP/HTTPS	etcd client	Patroni and etcd administration
2380	TCP/HTTPS	etcd peer	etcd members only
22	TCP/SSH	Administration / optional repository transport	Management network only

3.3 Capacity Baseline

Do not size production from the small backup sizes visible in a lab. Establish peak connections, transactions per second, database size, daily WAL generation, write latency, read latency, cache hit ratio, backup duration, restore

duration, replica catch-up time, and expected growth. Capacity must include headroom for a failover event where one node is unavailable.

4. Installation

4.1 Package Installation Pattern

Use the operating system's approved repositories. Package names vary by distribution, so the following is intentionally a pattern rather than a vendor-specific one-line installer.

```
# Example package groups - adapt to the approved Linux distribution
postgresql-17
postgresql-17-contrib
patroni
etcd
pgbackrest
haproxy
pgbouncer
keepalived

# Verify versions after installation
postgres --version
patroni --version
etcd --version
pgbackrest version
haproxy -v
pgbouncer --version
keepalived --version
```

4.2 Service Ownership

- Disable any distribution PostgreSQL unit that would independently start the same data directory; Patroni should own PostgreSQL lifecycle.
- Enable etcd, Patroni, HAProxy, PgBouncer, and Keepalived only after their configurations validate.
- Run services with dedicated least-privilege operating-system accounts where supported.
- Set file permissions so database keys, PgBouncer authentication files, Patroni credentials, and pgBackRest repository credentials are not world-readable.

5. Configuration

5.1 etcd Reference Configuration

For production, protect etcd client and peer traffic with TLS. Each member should have a unique certificate signed by the trusted cluster CA. The example below shows the important settings conceptually; adapt paths and service syntax to the installed etcd release.

```
# Member pgn1 - conceptual environment settings
ETCD_NAME=pgn1
ETCD_DATA_DIR=/var/lib/etcd
ETCD_LISTEN_CLIENT_URLS=https://70.80.90.110:2379,https://127.0.0.1:2379
ETCD_ADVERTISE_CLIENT_URLS=https://70.80.90.110:2379
ETCD_LISTEN_PEER_URLS=https://70.80.90.110:2380
ETCD_INITIAL_ADVERTISE_PEER_URLS=https://70.80.90.110:2380
ETCD_INITIAL_CLUSTER=pgn1=https://70.80.90.110:2380,pgn2=https://70.80.90.111:2380,pgn3=https://70.80.90.112:2380
ETCD_INITIAL_CLUSTER_STATE=new

# TLS options: trusted CA, member cert/key, peer cert/key,
# client-cert-auth=true, peer-client-cert-auth=true
```

5.2 Patroni Reference Configuration

Use one cluster scope on all members and a unique member name per node. The example enables PostgreSQL 17, SCRAM, replication slots, WAL archiving through pgBackRest, and pgBackRest as the preferred replica creation method with pg_basebackup as fallback.

```
scope: postgres_cluster
namespace: /service/
name: pgn1

restapi:
 listen: 70.80.90.110:8008
 connect_address: 70.80.90.110:8008
 # For production, configure certfile/keyfile and restrict network access.

etcd3:
 hosts:
  - 70.80.90.110:2379
  - 70.80.90.111:2379
  - 70.80.90.112:2379
 protocol: https
 cacert: /etc/patroni/tls/ca.crt
 cert: /etc/patroni/tls/patroni.crt
 key: /etc/patroni/tls/patroni.key
```

```
bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: "on"
        max_wal_senders: 10
        max_replication_slots: 10
        wal_keep_size: 1GB
        archive_mode: "on"
        archive_command: 'pgbackrest --stanza=main archive-push %p'
        password_encryption: scram-sha-256
  initdb:
    - encoding: UTF8
    - data-checksums
  pg_hba:
    - host all all 10.0.0.0/8 scram-sha-256
    - host replication replicator 70.80.90.0/24 scram-sha-256

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 70.80.90.110:5432
  data_dir: /var/lib/pgsql/17/data
  bin_dir: /usr/pgsql-17/bin
  authentication:
    superuser:
      username: postgres
      password: <POSTGRES_ADMIN_SECRET>
    replication:
```

```
    username: replicator
    password: <REPLICATION_SECRET>
  rewind:
    username: rewind_user
    password: <REWIND_SECRET>
 create_replica_methods:
  - pgbackrest
  - basebackup
 pgbackrest:
  command: /usr/bin/pgbackrest --stanza=main --delta restore
  keep_data: true
  no_params: true
 basebackup:
  max-rate: 100M
```

Create separate pgn2 and pgn3 files by changing name, restapi addresses, and postgresql.connect_address. Keep cluster scope, DCS endpoints, and cluster-wide bootstrap settings consistent.

5.3 pgBackRest Configuration

```
# /etc/pgbackrest/pgbackrest.conf
[main]
pg1-path=/var/lib/pgsql/17/data

[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=4
process-max=4
start-fast=y
log-level-console=info
log-level-file=detail

# Production:
# - use a dedicated repository host or object storage where appropriate
# - configure TLS/SSH/repository credentials securely
# - consider repo encryption and immutable/off-site protection
```

5.4 HAProxy Role-Aware Routing

HAProxy should not decide primary status by checking port 5432 alone. Use the Patroni REST /primary endpoint so the write backend accepts only the member that is both running as primary and holding the Patroni leader lock.

```
global
    log /dev/log local0
    maxconn 4000

defaults
    log global
    mode tcp
    timeout connect 5s
    timeout client  1m
    timeout server  1m

frontend postgres_write
    bind *:5000
    mode tcp
    default_backend patroni_primary

backend patroni_primary
    mode tcp
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 2s fall 3 rise 2 on-marked-down shutdown-sessions
    server pgn1 70.80.90.110:5432 check port 8008
    server pgn2 70.80.90.111:5432 check port 8008
    server pgn3 70.80.90.112:5432 check port 8008

frontend postgres_read
    bind *:5001
    mode tcp
    default_backend patroni_replicas

backend patroni_replicas
    mode tcp
    balance leastconn
```

```
  option httpchk GET /replica?lag=64MB
  http-check expect status 200
  server pgn1 70.80.90.110:5432 check port 8008
  server pgn2 70.80.90.111:5432 check port 8008
  server pgn3 70.80.90.112:5432 check port 8008
```

5.5 PgBouncer

Transaction pooling usually gives the highest connection reuse, but it changes session semantics. Applications using session-level advisory locks, temporary tables across transactions, session SET state, or features that assume a fixed backend must be tested. Use session pooling when application behavior requires it.

```
[databases]
appdb = host=127.0.0.1 port=5000 dbname=appdb

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 50
reserve_pool_size = 10
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
admin_users = pgbouncer_admin
stats_users = pgbouncer_stats
ignore_startup_parameters = extra_float_digits

# Production TLS example:
# client_tls_sslmode = require
# client_tls_key_file = /etc/pgbouncer/tls/server.key
# client_tls_cert_file = /etc/pgbouncer/tls/server.crt
# server_tls_sslmode = verify-full
# server_tls_ca_file = /etc/pgbouncer/tls/ca.crt
```

5.6 Keepalived

```
vrrp_script chk_db_endpoint {
  script "/usr/local/sbin/check-pg-endpoint.sh"
  interval 2
```

```
    timeout 2
    fall 2
    rise 2
    weight -50
}

vrrp_instance VI_POSTGRES {
    state BACKUP
    interface <NETWORK_INTERFACE>
    virtual_router_id 51
    priority 120            # use 110/100 on the other nodes
    advert_int 1

    # Prefer VRRPv3 and network-level controls. If unicast is required,
    # configure unicast_src_ip and unicast_peer for the local network.

    virtual_ipaddress {
        70.80.90.100/24
    }

    track_script {
        chk_db_endpoint
    }
}
#!/bin/sh
# /usr/local/sbin/check-pg-endpoint.sh
# Verify both PgBouncer and HAProxy write endpoint locally.
timeout 1 bash -c '</dev/tcp/127.0.0.1/6432' || exit 1
timeout 1 bash -c '</dev/tcp/127.0.0.1/5000' || exit 1
exit 0
```

6. Implementation

6.1 Step-by-Step Procedure

1. Validate hostnames, IPs, time synchronization, DNS, firewall paths, storage mounts, kernel/file-descriptor limits, package repositories, certificates, and backup repository access.
2. Build and secure the three-member etcd cluster. Confirm quorum and endpoint health before Patroni starts.

3. Install PostgreSQL 17 binaries but do not initialize or independently start the target PostgreSQL data directory if Patroni bootstrap will create it.
4. Install and configure pgBackRest. Create the stanza after PostgreSQL is available, validate archive configuration, and take the first full backup.
5. Create the Patroni YAML on each node with unique member names/addresses and consistent cluster scope/DCS configuration.
6. Start Patroni on the intended bootstrap node, then start Patroni on the remaining two nodes. Confirm one leader and two streaming replicas.
7. Configure HAProxy and validate that only the leader is UP in the write backend. Validate replica routing separately if used.
8. Configure PgBouncer, authentication, pool sizing, and TLS. Test application compatibility with the selected pool_mode.
9. Configure Keepalived and the VIP. Confirm only one node owns the VIP and that it moves when the tracked proxy endpoint fails.
10. Run functional, switchover, unplanned failover, backup, restore, replica rebuild, monitoring, and application reconnection tests before go-live.

6.2 Initial Cluster Validation

```
patronictl -c /etc/patroni/patroni.yml list

curl -s http://127.0.0.1:8008/patroni

psql -h 127.0.0.1 -p 5432 -U postgres -d postgres   -c "select pg_is_in_recovery(),
current_setting('server_version');"
```

Expected result: one Patroni member is Leader/primary and two are Replica/running. On the primary, pg_is_in_recovery() returns false; on replicas it returns true.

7. Validation

7.1 Database and Replication Validation

```
-- Run on the primary
SELECT application_name, client_addr, state, sync_state,
      pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_lag_bytes
FROM pg_stat_replication
ORDER BY application_name;


-- Run on every node
SELECT inet_server_addr(), pg_is_in_recovery(), now();


-- Write/read test through the VIP and PgBouncer
CREATE TABLE IF NOT EXISTS ha_validation(
 id bigserial PRIMARY KEY,
 created_at timestamptz DEFAULT now(),
 node inet DEFAULT inet_server_addr()
```

```
);
INSERT INTO ha_validation DEFAULT VALUES;
SELECT * FROM ha_validation ORDER BY id DESC LIMIT 5;
```

7.2 Routing Validation

```
# Primary endpoint must return 200 only on the current primary
curl -i http://70.80.90.110:8008/primary
curl -i http://70.80.90.111:8008/primary
curl -i http://70.80.90.112:8008/primary

# HAProxy config validation
haproxy -c -f /etc/haproxy/haproxy.cfg

# PgBouncer admin console examples
psql -h 127.0.0.1 -p 6432 -U pgbouncer_admin pgbouncer -c "SHOW POOLS;"
psql -h 127.0.0.1 -p 6432 -U pgbouncer_admin pgbouncer -c "SHOW STATS;"
```

7.3 VIP Validation

```
ip addr show <NETWORK_INTERFACE> | grep 70.80.90.100
ss -lntp | egrep ':(5000|5001|6432|8008|5432)\b'
systemctl status keepalived haproxy pgbouncer patroni
```

7.4 Switchover Test

Use switchover for planned maintenance because it starts from a healthy cluster and allows Patroni to coordinate leadership transfer.

```
# Review cluster first
patronictl -c /etc/patroni/patroni.yml list

# Interactive controlled switchover
patronictl -c /etc/patroni/patroni.yml switchover

# Validate immediately afterward
patronictl -c /etc/patroni/patroni.yml list
psql -h 70.80.90.100 -p 6432 -U <APP_USER> -d <DB>   -c "select inet_server_addr(), pg_is_in_recovery();"
```

8. Monitoring and Alerting

8.1 Minimum Monitoring

Layer	Monitor	Alert Examples

Layer	Monitor	Alert Examples
PostgreSQL	Availability, connections, TPS, latency, locks, deadlocks, checkpoints, WAL, replication	Primary unavailable, connection saturation, long transactions, replication lag, disk pressure
Patroni	Member state, role, timeline, DCS reachability, REST health	No leader, member stopped, repeated failover, DCS errors
etcd	Quorum, leader changes, fsync latency, DB size, peer/client TLS	Lost quorum, unhealthy endpoint, high disk latency, certificate expiry
pgBackRest	Last successful backup, backup age, WAL archive, repository capacity	Backup failure, stale backup, archive failure, repository low space
HAProxy	Backend state, sessions, errors, queue	No primary backend, flapping backend, maxconn pressure
PgBouncer	Pools, waiters, client/server connections, query/transaction counters	Waiting clients, pool exhaustion, high churn
Keepalived	VIP owner, VRRP transitions, tracked script	Unexpected VIP move, no VIP owner, repeated transitions
OS/Storage	CPU, memory, load, disk latency, filesystem, inode, network	Disk > threshold, high await, OOM, packet loss, clock drift

8.2 Patroni Monitoring Endpoint

```
curl -s http://127.0.0.1:8008/patroni | jq .

curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8008/primary

curl -s -o /dev/null -w '%{http_code}\n' 'http://127.0.0.1:8008/replica?lag=64MB'
```

8.3 Suggested Alert Threshold Approach

- Alert on loss of primary routing immediately; page if no writable endpoint remains after the agreed failover window.
- Alert on replica lag by both bytes and time, tuned to workload and RPO rather than a universal fixed value.
- Alert when backup age exceeds the backup schedule plus an operational grace period.
- Alert on sustained PgBouncer waiting clients, HAProxy backend flapping, etcd quorum risk, and storage latency.
- Track TLS certificate expiry for etcd, Patroni REST, PgBouncer, PostgreSQL, and repository transport.

9. Backup, Recovery and Disaster Recovery

9.1 Backup Operating Model

A highly available cluster is not a backup. Patroni protects service continuity; pgBackRest protects recoverability. Keep backup copies outside the database failure domain and test restore regularly.

```
# Create stanza once PostgreSQL/archive configuration is ready

sudo -u postgres pgbackrest --stanza=main stanza-create


# Validate repository and WAL archiving

sudo -u postgres pgbackrest --stanza=main check
```

```
# Full backup
sudo -u postgres pgbackrest --stanza=main --type=full backup

# Differential backup
sudo -u postgres pgbackrest --stanza=main --type=diff backup

# Review backup inventory
sudo -u postgres pgbackrest --stanza=main info
```

9.2 Restore Test

Perform restore testing on an isolated host or isolated data directory. Never overwrite an active production data directory unless executing an approved recovery procedure.

```
# Example isolated restore pattern
systemctl stop patroni
rm -rf <ISOLATED_PGDATA>/*
pgbackrest --stanza=main --pg1-path=<ISOLATED_PGDATA> restore

# Validate restored cluster with pg_controldata and an isolated PostgreSQL start.
# For PITR, configure the target time/LSN/name using the pgBackRest restore options
# appropriate to the selected pgBackRest release.
```

9.3 Replica Rebuild Using Patroni

```
# Reinitialize a failed/stale replica from the configured create_replica_methods.
# Confirm the target member carefully before execution.
patronictl -c /etc/patroni/patroni.yml reinit postgres_cluster pgn2
```

Because pgBackRest is first in create_replica_methods, Patroni can use it to seed the replica when the command succeeds; pg_basebackup remains the fallback in the reference configuration.

9.4 DR Design

- Define RPO and RTO before choosing backup frequency, WAL archive design, repository retention, and off-site replication.
- Maintain at least one backup copy outside the primary site/failure domain.
- Document DNS/VIP/application changes required if DR uses a separate network or site.
- Test restore of a full backup plus WAL to a target point, not only backup creation.
- Record measured restore time, WAL replay time, application validation time, and decision points for declaring DR.

10. Security and Hardening

☐ Use SCRAM-SHA-256 for PostgreSQL and PgBouncer password authentication where password auth is used.

☐ Restrict pg_hba.conf by source network, database, and role; do not use broad trust rules in production.

☐ Use TLS for PostgreSQL client connections when required by policy and verify server identity from clients.

☐ Protect Patroni REST API from untrusted networks; enable TLS and authentication/authorization controls where operationally appropriate.

☐ Use etcd TLS for client and peer traffic and require trusted client certificates in production.

☐ Keep replication, rewind, backup, monitoring, PgBouncer admin, and application roles separate with least privilege.

☐ Store secrets in an approved secret-management system; do not commit passwords to source control.

☐ Restrict HAProxy stats/admin sockets and PgBouncer admin console to management networks.

☐ Protect pgBackRest repository credentials and consider repository encryption plus immutable/off-site copies.

☐ Enable OS auditing/log forwarding and monitor privilege changes, authentication failures, configuration changes, and failover events.

☐ Patch the OS and components through controlled maintenance with tested rollback and switchover procedures.

10.1 PostgreSQL HBA Example

```
# Example only - replace networks with approved ranges
hostssl  appdb      app_user      10.20.0.0/16      scram-sha-256
hostssl  all        dba_role      10.30.10.0/24     scram-sha-256
hostssl  replication replicator   70.80.90.0/24     scram-sha-256
hostssl  postgres   rewind_user   70.80.90.0/24     scram-sha-256
```

11. High Availability and Failure Scenarios

11.1 Primary Node Failure

Stage	Action / Expected Behavior
Symptom	Application connections to the former primary fail; Patroni detects leader loss.
What to Check	patronictl list, Patroni logs, etcd health, HAProxy backend state, replica lag.
Expected Behavior	An eligible replica wins the leader race and is promoted. HAProxy /primary checks move write traffic to it.
Corrective Action	Restore/rebuild the failed member as a replica. Do not force the old primary back as writable.
Validation	Application writes succeed through VIP/PgBouncer; one leader exists; two replicas stream when recovery completes.

11.2 Replica Failure

Service should remain writable because the leader is unchanged. Investigate storage/network/PostgreSQL/Patroni errors, confirm WAL retention is sufficient, and use patronictl reinit if the replica cannot safely catch up.

11.3 etcd Member Failure

With three healthy etcd members, one member can fail without losing quorum. Do not remove or rebuild members casually; preserve quorum and follow the etcd member replacement procedure for the installed release.

11.4 Loss of etcd Quorum

Patroni cannot safely perform normal leader coordination without DCS quorum. Existing PostgreSQL may continue serving depending on state and configuration, but automated topology changes are unsafe. Treat quorum loss as a control-plane incident and restore etcd quorum before attempting failover operations.

11.5 HAProxy/PgBouncer Failure on VIP Owner

The Keepalived tracked script should lower the local VRRP priority or fault the instance so another proxy node takes the VIP. Validate that the replacement node has healthy PgBouncer and HAProxy before it becomes the client endpoint.

11.6 Backup Repository Failure

Database HA may remain available, but RPO begins to degrade if WAL cannot archive. Alert immediately on archive failures and repository capacity/availability. Do not ignore repeated archive_command failures.

12. Troubleshooting

12.1 Patroni Shows No Leader

```
patronictl -c /etc/patroni/patroni.yml list

journalctl -u patroni -n 200 --no-pager

etcdctl --endpoints=https://70.80.90.110:2379,https://70.80.90.111:2379,https://70.80.90.112:2379  --cacert=<CA> --cert=<CERT> --key=<KEY> endpoint health
```

Interpretation: first prove DCS quorum and network/TLS reachability. Then inspect Patroni member state and PostgreSQL startup errors. Avoid manual promotion until the cluster state and fencing risk are understood.

12.2 Replica Lag Increasing

```
SELECT application_name, client_addr, state,
    pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes
FROM pg_stat_replication;


# Replica-side checks
SELECT pg_is_in_recovery(),
    pg_last_wal_receive_lsn(),
    pg_last_wal_replay_lsn(),
    pg_last_xact_replay_timestamp();
```

Possible causes include network throughput, replica disk latency, long-running replay conflicts, resource saturation, WAL bursts, or a stopped receiver. Correct the bottleneck before reinitializing a replica.

12.3 pgBackRest Check Fails

```
pgbackrest --stanza=main --log-level-console=detail check

pgbackrest --stanza=main info

grep -E 'archive_command|archive_mode' /var/lib/pgsql/17/data/postgresql.conf
```

```
journalctl -u patroni -n 100 --no-pager
```

The pgBackRest check command validates repository/database configuration and, on a primary, can force WAL activity to confirm archiving. Fix archive/repository errors before trusting backup completeness.

12.4 PgBouncer Clients Waiting

```
psql -h 127.0.0.1 -p 6432 -U pgbouncer_admin pgbouncer -c "SHOW POOLS;"
psql -h 127.0.0.1 -p 6432 -U pgbouncer_admin pgbouncer -c "SHOW STATS;"
ss -s
ulimit -n
```

Check cl_waiting, server pool saturation, PostgreSQL max_connections, file-descriptor limits, slow transactions, and whether pool sizing matches the number of database/user pools.

12.5 VIP Does Not Move

```
systemctl status keepalived
journalctl -u keepalived -n 200 --no-pager
/usr/local/sbin/check-pg-endpoint.sh; echo $?
ip addr show <NETWORK_INTERFACE>
tcpdump -ni <NETWORK_INTERFACE> proto 112
```

Check VRRP network reachability, interface names, virtual_router_id, priorities, unicast/multicast design, firewall rules, and the tracked script result.

13. Maintenance and Upgrade Considerations

- Use planned Patroni switchover before patching the current primary.
- Patch one replica at a time, validate it rejoins and catches up, then proceed to the next node.
- Keep PostgreSQL major upgrades separate from routine package patching. Major upgrades require an approved upgrade method and rollback plan.
- Review Patroni release notes before changing Patroni major/minor versions and validate configuration compatibility.
- Upgrade etcd conservatively while maintaining quorum and following the release-specific supported upgrade path.
- After HAProxy/PgBouncer/Keepalived changes, validate configuration syntax before reload/restart.
- Re-run backup check and a restore test after significant storage, repository, TLS, or PostgreSQL changes.
- Maintain a configuration baseline in version control without secrets.

13.1 Maintenance Sequence

11. Confirm healthy cluster, current backups, no excessive replication lag, and no unresolved alerts.
12. Switchover away from the node to be maintained if it is primary.
13. Stop/patch/reboot one node only.
14. Start services and verify Patroni role, streaming state, HAProxy health, PgBouncer, and monitoring.
15. Wait for replica lag to return to normal before touching another node.
16. At completion, perform an application write/read test through the VIP and record the change evidence.

14. Production Readiness and Go-Live Checklists

14.1 Pre-Implementation Checklist

☐ Architecture and failure domains approved.

☐ RPO/RTO and backup retention approved.

☐ Capacity and growth model approved.

☐ VIP, DNS, ports, firewall, certificates, and service accounts approved.

☐ Monitoring dashboards, alert routes, on-call ownership, and escalation defined.

☐ Rollback plan and maintenance window approved.

14.2 Configuration Checklist

☐ Unique Patroni member names and addresses configured.

☐ Same Patroni scope/DCS cluster used on all members.

☐ PostgreSQL 17 binary/data paths verified.

☐ SCRAM credentials and pg_hba rules validated.

☐ pgBackRest archive_command, stanza, repository, retention, and permissions validated.

☐ HAProxy write health check uses Patroni /primary or /read-write, not only TCP 5432.

☐ PgBouncer pool_mode validated with application behavior.

☐ Keepalived tracks the actual client path and only one node owns the VIP.

14.3 HA / DR Checklist

☐ Planned switchover tested.

☐ Unplanned primary failure tested.

☐ Replica failure and rebuild tested.

☐ Proxy/VIP owner failure tested.

☐ Single etcd member failure tested.

☐ Backup creation and pgBackRest check tested.

☐ Isolated full restore tested.

☐ Point-in-time recovery tested where required by RPO.

☐ Off-site/secondary repository recovery path tested.

☐ Measured RTO recorded and accepted.

14.4 Go-Live Checklist

☐ Cluster has exactly one leader and expected replicas.

☐ Replication lag is within the approved threshold.

☐ VIP is owned by the intended proxy node.

☐ Application connects only through approved endpoint(s).

☐ Write test and read test pass.

☐ Backup is current and WAL archive is healthy.

☐ Monitoring and alerting are green.

☐ Runbook, credentials access, support contacts, and escalation are available to operations.

☐ Change record includes final configurations, validation evidence, and rollback status.

14.5 Post-Implementation Validation Checklist

☐ 24-hour stability review completed after go-live.

☐ No repeated Patroni leader changes or Keepalived VRRP flaps.

☐ No sustained PgBouncer waiters or HAProxy backend flapping.

☐ No PostgreSQL archive failures, replication errors, or abnormal disk latency.

☐ Backup completed on schedule and repository capacity trend is normal.

☐ Operational team has completed at least one supervised switchover and restore exercise.

15. Operational Commands / Cheat Sheet

Purpose	Command
Cluster status	patronictl -c /etc/patroni/patroni.yml list
Patroni node JSON	curl -s http://127.0.0.1:8008/patroni | jq .
Primary health	curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8008/primary
PostgreSQL role	psql -c "select pg_is_in_recovery();"
Replication	psql -c "select application_name,state,sync_state from pg_stat_replication;"
pgBackRest status	pgbackrest --stanza=main info
pgBackRest validation	pgbackrest --stanza=main check
HAProxy syntax	haproxy -c -f /etc/haproxy/haproxy.cfg
PgBouncer pools	psql -p 6432 -U pgbouncer_admin pgbouncer -c "show pools;"
VIP owner	ip addr show <NETWORK_INTERFACE> | grep <VIP>
Service logs	journalctl -u patroni -u haproxy -u pgbouncer -u keepalived --since '-15 min'

16. Key Takeaways

- Patroni provides database HA orchestration; it does not replace backups.
- HAProxy must route by Patroni role health, not merely by an open PostgreSQL port.
- PgBouncer protects PostgreSQL from excessive connection concurrency but its pooling mode must match application semantics.
- Keepalived provides a stable network endpoint; track the actual proxy/pool path so a broken node does not retain the VIP.
- etcd quorum is part of the HA control plane and must be secured, monitored, and operated as carefully as the database.
- Production readiness is proven by failover and restore testing, not by service status alone.

17. Official References

PostgreSQL 17 - Replication configuration: https://www.postgresql.org/docs/17/runtime-config-replication.html

PostgreSQL 17 - pg_hba.conf: https://www.postgresql.org/docs/17/auth-pg-hba-conf.html

Patroni - Documentation: https://patroni.readthedocs.io/en/latest/

Patroni - REST API: https://patroni.readthedocs.io/en/latest/rest_api.html

Patroni - Replica imaging and bootstrap: https://patroni.readthedocs.io/en/rel_3_3/replica_bootstrap.html

pgBackRest - User Guide: https://pgbackrest.org/user-guide.html

HAProxy - Health checks: https://www.haproxy.com/documentation/haproxy-configuration-tutorials/reliability/health-checks/

HAProxy - TCP proxying: https://www.haproxy.com/documentation/haproxy-configuration-tutorials/protocol-support/tcp/

PgBouncer - Configuration: https://www.pgbouncer.org/config

Keepalived - Configuration Synopsis: https://www.keepalived.org/documentation/user-guide/configuration-synopsis/

etcd - v3.7 Documentation: https://etcd.io/docs/v3.7/

etcd - Transport Security: https://etcd.io/docs/v3.6/op-guide/security/

Appendix A - Complete Reference Configuration

A.1 Patroni Per-Node Changes

Setting	pgn1	pgn2	pgn3
name	pgn1	pgn2	pgn3
restapi.listen/connect_address	70.80.90.110:8008	70.80.90.111:8008	70.80.90.112:8008
postgresql.connect_address	70.80.90.110:5432	70.80.90.111:5432	70.80.90.112:5432
Keepalived priority	120	110	100

A.2 Safe Validation Script

```
#!/usr/bin/env bash
set -u

CFG=/etc/patroni/patroni.yml
VIP=70.80.90.100

echo "=== Patroni ==="
patronictl -c "$CFG" list || true

echo "=== Local Patroni REST ==="
curl -fsS http://127.0.0.1:8008/patroni || true
echo

echo "=== PostgreSQL listeners ==="
ss -lnt | egrep ':(5432|6432|5000|5001|8008)\b' || true

echo "=== VIP ==="
```

```
ip -brief addr | grep "$VIP" || echo "VIP not local on this node"

echo "=== pgBackRest ==="
sudo -u postgres pgbackrest --stanza=main info || true

echo "=== Services ==="
systemctl --no-pager --full status patroni haproxy pgbouncer keepalived 2>/dev/null | \
  egrep 'Loaded:|Active:' || true
```

A.3 Failover Evidence Collection

```
#!/usr/bin/env bash
OUT="/var/tmp/pg-ha-evidence-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$OUT"

patronictl -c /etc/patroni/patroni.yml list > "$OUT/patroni-list.txt" 2>&1
curl -s http://127.0.0.1:8008/patroni > "$OUT/patroni-local.json" 2>&1
ip addr > "$OUT/ip-addr.txt" 2>&1
ss -lntp > "$OUT/listeners.txt" 2>&1
journalctl -u patroni -u haproxy -u pgbouncer -u keepalived \
  --since "-30 min" --no-pager > "$OUT/services.log" 2>&1
sudo -u postgres pgbackrest --stanza=main info > "$OUT/pgbackrest-info.txt" 2>&1

echo "Evidence saved to $OUT"
```

Appendix B - Validation and Test Scripts

B.1 End-to-End Connectivity Test

```
#!/usr/bin/env bash
set -euo pipefail

HOST="${1:-70.80.90.100}"
PORT="${2:-6432}"
DB="${3:-appdb}"
USER="${4:-app_user}"

psql "host=$HOST port=$PORT dbname=$DB user=$USER connect_timeout=5" \
  -v ON_ERROR_STOP=1 <<'SQL'
```

```
SELECT now() AS checked_at,
       inet_server_addr() AS backend_ip,
       pg_is_in_recovery() AS is_replica;

CREATE TABLE IF NOT EXISTS ha_smoke_test(
    id bigserial PRIMARY KEY,
    created_at timestamptz NOT NULL DEFAULT now(),
    backend_ip inet NOT NULL DEFAULT inet_server_addr()
);

INSERT INTO ha_smoke_test DEFAULT VALUES;
SELECT * FROM ha_smoke_test ORDER BY id DESC LIMIT 3;
SQL
```

B.2 Backup Validation Checklist Script

```
#!/usr/bin/env bash
set -euo pipefail

STANZA="${1:-main}"

echo "1) pgBackRest check"
sudo -u postgres pgbackrest --stanza="$STANZA" check

echo "2) Backup inventory"
sudo -u postgres pgbackrest --stanza="$STANZA" info

echo "3) PostgreSQL archive status"
sudo -u postgres psql -XAtqc "
SELECT archived_count, failed_count, last_archived_wal,
       last_archived_time, last_failed_wal, last_failed_time
FROM pg_stat_archiver;"

echo "PASS: review timestamps, retention, and repository capacity before closing."
```